Unité 4 — Cahier de laboratoire (étudiant)

Livrable C — Défenses & contournements · UE « Software Security »

Code UE 7532 14 U32 D1 · Document étudiant · 11 octobre 2026

Cadre. Environnement isolé (livrable E).

Lab 4.1 — Observer les protections & déclencher le canary EXERCICE GUIDÉ

1. Objectifs

- Lire les protections d'un binaire ; constater l'effet du canary.

2. Prérequis

- Unité 2 ; Cours U4 §2,7.

3. Environnement

Image (E). Réutilise `stack3.c` (Unité 2).

4. Fichiers

`stack3.c` (Unité 2), recompilé avec/sans protections.

5. Commandes

SHELL

```
gcc -m32 -fno-pie -no-pie -fstack-protector-all -g stack3.c -o stack3_canary
readelf -lW stack3 | grep GNU_STACK          # RWE (labo) vs RW (durci)
readelf -s stack3_canary | grep stack_chk     # canary present ?
./stack3_canary "$(python3 -c 'print("A"*80)')"   # debordement
```

6. Manipulations

a. Comparez `GNU_STACK` et la présence de `__stack_chk_fail` entre le binaire de l'Unité 2 et la version durcie.

b. Débordez la version avec canary : observez l'abandon « stack smashing detected ».

7. Résultats attendus

Le binaire durci affiche `RW` (NX) et un canary ; le débordement est stoppé par un abandon (code 134).

8. Explications

Le canari placé avant l'adresse de retour est écrasé par le débordement ; sa vérification échoue → abandon avant le `ret`.

9. Questions

1. Quelle différence `GNU_STACK` entre les deux binaires ?
2. Pourquoi l'exploit de l'Unité 2 ne fonctionne-t-il plus ici ?
3. Comment un attaquant pourrait-il contourner le canary ?

11. Vulnérabilité & remédiation

Défense : stack canary + NX. **Limite :** fuite du canari, ret-to-libc (Lab 4.2).

12. Durée

1 h.

10. Corrigé : livrable D, Lab 4.1.

Lab 4.2 — NX et introduction à ROP EXERCICE GUIDÉ

1. Objectifs

- Constater que le shellcode en pile échoue sous NX ; découvrir les gadgets ROP.

2. Prérequis

- Lab 4.1 ; Unité 2 (exploit).

3. Environnement

Image (E). Réutilise `stack3.c` (Unité 2).

4. Fichiers

`stack3.c` (Unité 2), recompilé avec/sans protections.

5. Commandes

SHELL

```
gcc -m32 -fno-pie -no-pie -fno-stack-protector -g stack3.c -o stack3_nx   # NX actif (pas de -z execstack)
readelf -lW stack3_nx | grep GNU_STACK        # RW
# rejouer l'exploit shellcode de l'Unite 2 -> echoue (pile non executable)
# chercher des gadgets pour une approche ROP :
python3 -m ropper --file stack3_nx --nocolor --search "pop"
```

6. Manipulations

a. Compilez sans `-z execstack` : la pile est `RW`.

b. Rejouez l'exploit « jmp esp + shellcode » de l'Unité 2 : il ne donne plus de shell (code non exécutable).

c. Listez des gadgets avec `ropper` : comprenez comment les enchaîner (ret-to-libc / ROP).

7. Résultats attendus

Sous NX, le shellcode en pile ne s'exécute pas ; `ropper` liste des gadgets (`pop … ; ret`) réutilisables.

8. Explications

NX empêche l'exécution de données ; la parade est de réutiliser du code exécutable existant (libc, gadgets) plutôt que d'en injecter.

9. Questions

1. Pourquoi le shellcode en pile échoue-t-il ?

2. Qu'est-ce qu'un gadget ? Donnez-en deux issus de ropper.
3. En quoi ret-to-libc contourne-t-il NX ?

11. Vulnérabilité & remédiation

Défense : NX. **Contournement :** ret-to-libc / ROP. **Défense ultérieure :** ASLR + PIE.

12. Durée

1 h 30.

10. Corrigé : livrable D, Lab 4.2.

Lab 4.3 — Durcir un binaire et conclure EXERCICE AUTONOME

1. Objectifs

- Comparer un binaire durci (toutes protections) à la version de l'Unité 2.

2. Prérequis

- Labs 4.1–4.2.

3. Environnement

Image (E). Réutilise `stack3.c` (Unité 2).

4. Fichiers

`stack3.c` (Unité 2), recompilé avec/sans protections.

5. Commandes

SHELL

```
gcc -m32 -fstack-protector-all -g stack3.c -o stack3_hard   # PIE + canary + NX (defauts modernes)
readelf -lW stack3_hard | grep GNU_STACK
checksec --file=stack3_hard 2>/dev/null || readelf -d stack3_hard | grep -i bind_now
```

6. Manipulations

a. Compilez une version durcie (laisser PIE/NX par défaut + canary).

b. Dressez un tableau comparatif des protections (labo vs durci).

c. Rédigez une courte conclusion sur la défense en profondeur.

7. Résultats attendus

Le binaire durci cumule canary, NX, PIE (et RELRO) ; l'exploitation de l'Unité 2 n'y est plus directement applicable.

8. Explications

Chaque protection ferme une étape ; leur combinaison impose à l'attaquant fuites et chaînes complexes.

9. Questions

1. Quelles protections sont actives sur le binaire durci ?
2. Quelle(s) technique(s) resteraient envisageables, et à quelle condition ?
3. Quel est l'intérêt d'un langage memory-safe (U6) ?

11. Vulnérabilité & remédiation

Synthèse : défense en profondeur ; la suppression à la source passe par U6/U9.

12. Durée

1 h.

10. Corrigé : livrable D, Lab 4.3.

Défenses ROP ; sorties vérifiées.

U04 — Correctifs techniques vérifiés

Annexe de correction au document original — les sections non citées restent inchangées

Lab 4.2 — Remplacement des consignes de rejeu de l exploit U02

La comparaison directe avec test_shellcode.c remplace le rejeu non vérifié de exploit_stack3.py.

Depuis /workspace/src, compiler les deux variantes :

```
gcc -m32 -fno-stack-protector -fno-pie -no-pie -z noexecstack -g \
  test_shellcode.c -o test_shellcode_nx
gcc -m32 -fno-stack-protector -fno-pie -no-pie -z execstack -g \
  test_shellcode.c -o test_shellcode_exec
readelf -lW test_shellcode_nx | grep GNU_STACK
readelf -lW test_shellcode_exec | grep GNU_STACK
./test_shellcode_nx
echo "Code de sortie : $?"
./test_shellcode_exec
```

Dans le shell ouvert : echo NX_DESACTIVE_OK ; puis exit.

NX actif : GNU_STACK RW, SIGSEGV, code 139. NX inactif : RWE, shell opérationnel.

Lab 4.2 — Recherche de gadgets (consigne conservée)

```
python3 -m ropper --file stack3_nx --nocolor --search "pop"
```

Observer les gadgets trouvés ; leurs adresses peuvent changer selon la compilation.

Lab 4.3 — Complément pour vérifier le Full RELRO

```
readelf -lW stack3_hard | grep GNU_RELRO
readelf -dW stack3_hard | grep -E 'BIND_NOW|FLAGS'
```

GNU_RELRO + BIND_NOW permettent de conclure à Full RELRO dans ce contexte.

Portée des validations

Canary : arrêt 134 ; NX : arrêt 139 ; pile exécutable : shell ; Ropper : gadgets listés.

Un exploit ROP complet et le rejeu de exploit_stack3.py ne sont pas validés ici.